COSC2637 - Big Data Processing

Lecture 12
Course Review
End of Semester Survey
Timed Assessment

RMIT UNIVERSITY

Week 2 - Hadoop, HDFS, YARN

- What is clouding computing?
  - Web 2.0; Utility computing; Everything as a service (IaaS, PaaS, Saas)
  - Ready for big data problem
  - Data center and energy consumption
- Apache Hadoop
  - HDFS
    - Write once, read many; Highly fault tolerant, …
    - Master/slave architecture, DataNode, NameNode, Namespace, heartbeat, blockreport
    - Data Replication, block, placement, pros and cons to place replicas on unique racks.
  - YARN
    - Hadoop 1.0, Scalability bottleneck, Hadoop 2.0
    - Resource Manager, Scheduler, Application Manager, Node Manager, Application Master, Container,
    - Application workflow in Hadoop YARN.

Week 3 - MapReduce Basics

- Typical Big Data Problem
- Power of MapReduce
- MapReduce Execution Platform – handle everything else
  - Scheduling
    - jobs, fair scheduler, straggler, two causes of straggler, Zipfian distribution
  - data distribution
    - Data and code co-location
  - synchronization,
    - Enforce proper execution order, map tasks, reduce tasks, shuffle and sort
  - errors and faults
    - Hardware failure, software failure
- Map, Reduce, Combinate, Partition
- Shuffle and Sort
- As a programmer, what you can control, what you don't know.

Week 3 - MapReduce Basics (cont.)

- Local Aggregation
  - Why local aggregation
  - Three ways
    - Combiner
      - Hadoop makes no guarantees on how many time combiners are called, it could be 0, 1, or multiple times
      - Mapper output = combiner input = combiner output = reducer input
    - In-mapping combining
      - advantage, disadvantage
    - In-mapping combining (preserve state across lines)
  - Local aggregation algorithm correctness
    - 5 solutions

Week 4 - MapReduce Co-occurrence Matrix

- Two MapReduce Algorithms
  - Pairs Approach
  - Stripes Approach – Associative Array – More efficient
  - Combiner, in-mapper combining
- Co-occurrence Relative Frequencies
  - Stripes Approach
    - Easy! memory requirement
  - Pairs Approach
    - Partitioner is required.
    - Count the total number of the first word for the key with two words
      - keep all pairs in reducer memory – memory requirement
      - Insert a dummy pair for each key-value pair in the mapper stage – no memory requirement.

Week 5 - Clustering

- Similarity or Distance Measure
  - metrics, norms, cosine, Jaccard ratio
- Hierarchical Agglomerative Clustering (HAC)
  - The similarity between two clusters
    - single Link, complete link, group average
- K-means Clustering

Lloyd's Algorithm

1. Select k random point $\{s_1, s_2, \ldots s_k\}$ as seeds.
2. Until clusters converge:
   - **Assign** each point x_i to the cluster c_j such that $d(x_i, s_j)$ is minimal
   - For each point x_i, find **nearest centroid** c_j, assign the point x_i to cluster j
   - **Refine** the seeds to the centroid of each cluster
   - For each cluster, the centroid is updated on each dimension k:

$$c_j^{(t+1)} = \frac{1}{|X_j^{(t)}|} \sum_{x \in X_j^{(t)}} x$$

If not converged

Stop when none of the cluster assignments change – i.e., converged

Week 5 - Clustering

- K-means Clustering
  - Knee/elbow finding
- Others
  - number of mapper tasks
  - number of reducer tasks

Week 6 - Processing Large Graphs

- Two common representations of G = (V, E)
  - Adjacency matrix
  - Adjacency list
- Single-Source Shortest Path
  - Dijkstra's Algorithm
    - Dijkstra's Algorithm cannot be used in MapReduce Framework?
  - Parallel Breadth-First search (BFS)
    - MapReduce Algorithm
      - Initialization
      - Iterations
      - Stopping Criterion

Week 7 HIVE and PIG

- MapReduce is a major backwards to database community
  - Database access; No index; Incompatible with DBMS tools; others …
- Common idea of HIVE and PIG
  - Higher level language to facilitate large-data processing
  - Higher level language compiles down to Hadoop jobs
- HIVE - Facebook
  - HiveQL, a variant of SQL: HiveQL executes on Hadoop
  - select * from .. where …
  - HIVE Architecture
    - Metastore
- PIG – Yahoo!
  - PIG Latin, a dataflow language
  - Field, Bag, Tuple, Relation, Map
  - Understand a PIG script
  - Execution modes: local and mapreduce
  - PIG script execution mechanisms: interactive, batch, embedded

Week 8 Mahout, Solr, Storm

- Mahout
  - scalable machine learning algorithms
  - recommendation problem
- Solr
  - Enable create search engines searching websties, databases and files
  - Inverted index, Schema, Lucene query parser
- Storm
  - reads raw stream of real-time data from one end and passes it through a sequence of small processing units and output the processed/useful information at the other end.
  - Bolt, Spout, Stream
  - Topology

Week 9 -Introduction of Apache Spark

- How Spark runs on clusters
  - SparkContext, Cluster Manager, Task, Executor
- Spark RDD (Resilient Distributed Dataset)
  - RDD Operations
    - Transformations (map, filter, union, intersection …)
      - Spark Transformation is a function
      - Transformations are lazy - Directed Acyclic Graph (DAG) - DAG vs. RDD lineage
      - RDD lineage – data resilient, why?
    - Actions (count, taken, collect, …)
      - The values of action are stored to drivers or to the external storage system.
      - It brings laziness of RDD transformations into motion
      - The execution DAG or physical execution plan is the DAG of stages

Week 10 - Scala

- What is Scala?
  - object-oriented language, functional
- Scala Programming Basics
  - Declare immutable variable, mutable variable
  - Scala Function Definitions
  - Scala - Anonymous Function (Lambda Function)
  - Scala – Closures
  - Scala – Traits
  - Scala - Files I/O
  - Scala - Classes & Objects
  - The ‘s’ String Interpolator
  - The ‘ f ’ Interpolator
- Scala Practice – In-class exercise

Week 11 - Spark Streaming and Kafka

Spark Streaming

- DStream (Discretized Stream) - at each batch interval of a DStream, the source pulls in data and converts it to regular RDDs using the block store
- Input – basic sources (file systems) and advanced sources (Kafka, HDFS)
- Output – HDFS, Dashboard, Databases

• Spark Streaming Operations

- Transformation
  - RDD Transformations – updateStateByKey (state-based operation), map, flatmap, …
  - Window Operation – reduceByKeyAndWindow
- Action (Output Operations)
  - Design Patterns for using foreachRDD

• Caching / Persistence

- allow developers to persist the stream's data in memory
- automatically persisted in memory –
  - window-based operations state-based operations

Week 11 - Spark Streaming and Kafka (cont.)

- Checkpointing
  - Metadata checkpointing
  - Data checkpointing
  - When to enable Checkpointing
  - How to configure Checkpointing
- Shared Variables - accumulator and broadcast

Apache Kafka

- What is a Messaging System?
- Two types of messaging patterns are available:
  - point to point
  - publish-subscribe
- Apache Kafka – Fundamentals
- ZooKeeper service

Questions?

Timed Assessment – A4

A4 is a time-controlled assessment to evaluate the understanding of students on the fundamental and advanced knowledge in big data processing taught in the course.

- Weighting 25%, Nov 2 Monday, Online
- This is a 2-hour assessment that may be taken at any time within a 24-hour period (12:01AM Nov 2 – 12:01AM Nov 3)

Timed Assessment

Question Types

- Q1 Scala code understanding (2 points)
- Q2 MapReduce code understanding (2.5 points)
- Q3 MapReduce Q&A (3 points)
- Q4 Scala code understanding (3 points)
- Q5 MapReduce Q&A (2.5 points)
- Q6 MapReduce code understanding (4 points)
- Q7 Apache Pig code understanding (4 points)
- Q8 MapReduce coding (4 points)

Timed Assessment

Question Types

- Q1 Scala code understanding (2 points)
- Q2 MapReduce code understanding (2.5 points)
- Q3 MapReduce Q&A (3 points)
- Q4 Scala code understanding (3 points)
- Q5 MapReduce Q&A (2.5 points)
- Q6 MapReduce code understanding (4 points)
- Q7 Apache Pig code understanding (4 points)
- Q8 MapReduce coding (4 points)

- Using Chatgpt is not allowed
- Answers must be based on the knowledge taught in this courses
- No AWS EMR cluster is needed.

Timed Assessment

Lockdown Browser

- Browser used by educational institutions to prevent cheating during online exams

- Using Chatgpt is not allowed
- Answers must be based on the knowledge taught in this courses
- No AWS EMR cluster is needed.

Timed Assessment

Lockdown Browser

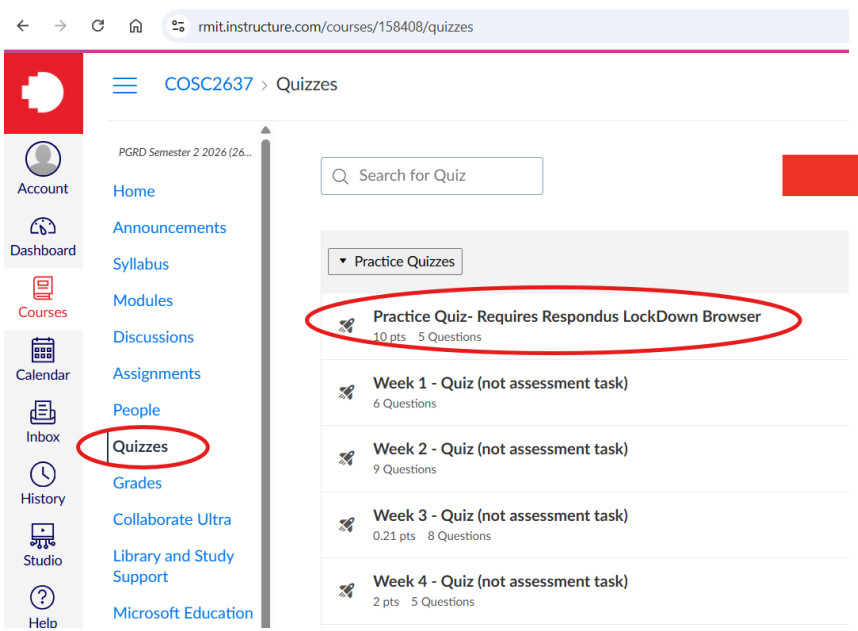
- Browser used by educational institutions to prevent cheating during online exams
- Make sure you install Lockdown Browser and know how to use it before sitting A4 by working on the following Quiz

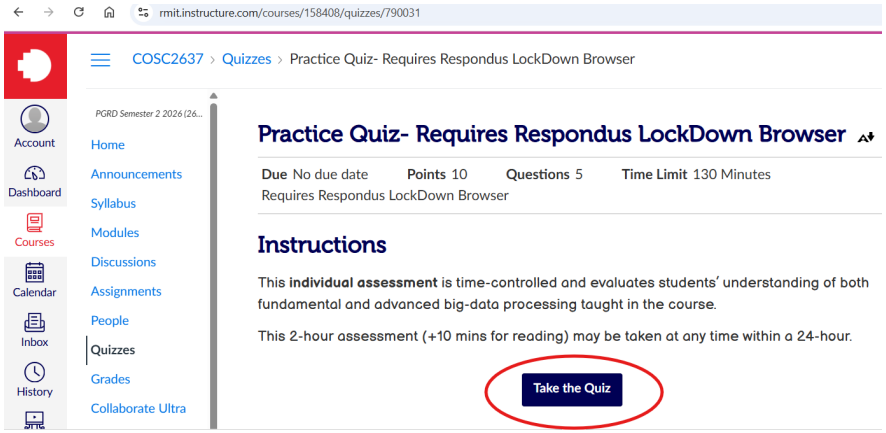
- Using Chatgpt is not allowed
- Answers must be based on the knowledge taught in this courses
- No AWS EMR cluster is needed.

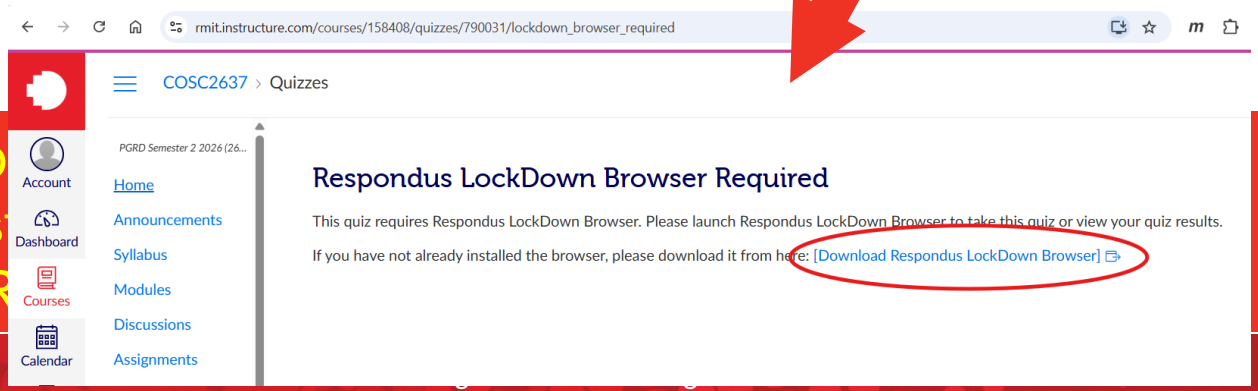
Timed Assessment

Lockdown Browser

- Browser used by educational institutions to prevent cheating during online exams
- Make sure you install Lockdown Browser and know how to use it before sitting A4 by working on the following Quiz







- Using Chatgp
- Answers mus
- No AWS EMR

Code understanding questions aims to evaluate the understanding of student to given code segments

Question Types

- Q1 Scala code understanding (2 points)
- Q2 MapReduce code understanding (2.5 points)
- Q3 MapReduce Q&A (3 points)
- Q4 Scala code understanding (3 points)
- Q5 MapReduce Q&A (2.5 points)
- Q6 MapReduce code understanding (4 points)
- Q7 Apache Pig code understanding (4 points)
- Q8 MapReduce coding (4 points)

Code understanding questions aims to evaluate the understanding of student to given code segments

Q&A question evaluates the understanding of student in MapReduce design and implementation

Question Types

- Q1 Scala code understanding (2 points)
- Q2 MapReduce code understanding (2.5 points)
- Q3 MapReduce Q&A (3 points)
- Q4 Scala code understanding (3 points)
- Q5 MapReduce Q&A (2.5 points)
- Q6 MapReduce code understanding (4 points)
- Q7 Apache Pig code understanding (4 points)
- Q8 MapReduce coding (4 points)

RMIT Classification: Trusted

Code understanding questions aims to evaluate the understanding of student to given code segments

Q&A question evaluates the understanding of student in MapReduce design and implementation

Question Types

- Q1 Scala code understanding (2 points)
- Q2 MapReduce code understanding (2.5 points)
- Q3 MapReduce Q&A (3 points)
- Q4 Scala code understanding (3 points)
- Q5 MapReduce Q&A (2.5 points)
- Q6 MapReduce code understanding (4 points)
- Q7 Apache Pig code understanding (4 points)
- Q8 MapReduce coding (4 points)

Coding questions evaluates the solution design of student

- Using pseudocode to show processing flow in MapReduce jobs
- Python code is not required
- Marking concerns the correctness of process flow in MapReduce jobs

Timed Assessment

Question Sample – Given two files on HDFS

A sample of Taxis.txt	A sample of Trips.txt
Taxi#, company, model, year	**Trip#, Taxi#, fare, distance, pickup_x, pickup_y, dropoff_x, dropoff_y**
470,0,80,2018	0,354,232.64,127.23,46.069,85.566,10.355,4.83
332,11,88,2013	1,173,283.7,150.74,5.02,31.765,88.386,27.265
254,10,62,2018	2,8,83.84,43.17,63.269,33.156,92.953,60.647
460,4,90,2022	3,340,259.2,136.3,14.729,13.356,14.304,90.273
113,6,23,2015	4,32,270.07,152.65,27.965,13.37,77.925,62.82
275,16,13,2015	5,64,378.31,202.95,1.145,94.519,98.296,35.469
318,14,46,2014	6,480,235.98,121.23,66.982,66.912,5.02,31.765
	7,410,293.16,162.29,2.841,95.636,91.029,16.232

The task is to write a MapReduce program with Python to count the number of trips for each taxi company. Marking is based on the logical flow of the solution. You need not write a complete Python-based MapReduce program. Instead, you can provide your solution using Pseudo-code.

Sample Solution

Mapper-1

```
for each line
    valuesinline = split by ","
    if valueinline has 4 values
        emit(taxi#, "Taxis.txt", company)        // this is a line from Taxis.txt
    else
        emit(taxi#, "Trips.txt")                 // this is a line from Trips.txt
```

Reducer-1

```
for each taxi#                                   //taxi# is the key
   for each input line
      if the input line has "Taxis.txt"
          extract company
      else the input line has "Trips.txt"
          count the number of lines for taxi#     //count the number of trips for taxi#
   output(company, trip count for taxi#)
```

Sample Solution

Mapper-2

for each line

valuesinline = split by “,”

emit(company, trip count for taxi#) //company is the key, trip count is value

Reducer-2

for each company

count the total value

output(company, total value) // total value is the trip number of the company across different taxis#.

Sample Solution

Mapper-2

for each line

valuesinline = split by “,”

emit(company, trip count for taxi#) //company is the key, trip count is value

Reducer-2

for each company

count the total value

output(company, total value) // total value is the trip number of the company across different taxis#.

No need to provide run.sh. But if partitioner or comparator is used, you must explain how it is customised and what they want to achieve.

Timed Assessment

How to prepare?

- Revisit the questions in A1, A2 and A3
- Go through the weekly lectorials and quizzes

Consultation Time

Consultation:

- To assist you to prepare A4, consultations are provided
  - Office 14-08-06, 10:30 – 12:30 AM Monday in week 14
  - Book a flexible online consultation with Ke via ke.deng@rmit.edu.au

Thanks for your support in this semester!

